

Identity and Profile Management API

Preliminary Report

Luca Cipriani

2025-12-15

Table of contents

1	Introduction	3
1.1	Project Concept	3
1.2	Problem Motivation	3
1.3	Project Scope	4
1.4	Why Existing Solutions Fail	4
2	Literature Review	5
2.1	The Technical and Cultural Challenge of Names	5
2.2	Identity Theory and Context Collapse	6
2.3	Privacy Regulations and Security Requirements	8
2.4	Critical Evaluation of Existing Solutions	9
3	Design	10
3.1	Problem Formulation and Theoretical Foundation	10
3.2	Domain and User Requirements	10
3.3	System Architecture	11
3.4	Feature Design and Justification	11
3.5	Security Architecture	11
3.6	Technology Stack	12
3.7	Comparison with Existing Solutions	12
3.8	System Context	13
3.9	Project Timeline	13
3.10	Implementation Plan	13
4	Feature Prototype: Context-Dependent Identity	16
4.1	Overview and Theoretical Foundation	16
4.2	Technical Implementation	16
4.3	Technical Challenges Addressed	17
4.4	Demonstration of Effectiveness	17
4.5	Evaluation Methodology	18
4.6	Evaluation Results	19
4.7	Proposed Improvements	19
4.8	Conclusion on Prototype	20
	Appendix: Data Model	21
4.9	Entity Relationship Diagram	21
4.10	JSONB Data Examples	21
	References	24

1 Introduction

1.1 Project Concept

Modern identity systems get authentication right but identity wrong. They can verify who you are, but they cannot represent the fact that you might go by “Dr. Smith” at work, “Sarah” with friends, and your Chinese name with family. They assume names follow Western conventions, that people have one fixed identity, and that privacy means hiding information rather than controlling how it flows.

This project implements a **context-aware identity and profile management API** that addresses a fundamental architectural gap in contemporary identity systems. The system treats identity as contextual, culturally diverse, and user-controlled rather than singular, Western-centric, and administratively fixed.

The implementation follows the **Web API Development** project template, focusing on RESTful API design, secure authentication and authorization, database modeling for complex domain requirements, and third-party integration via OAuth 2.1. The final deliverable consists of a fully functional backend API with a demonstration frontend deployed to production infrastructure.

1.2 Problem Motivation

Every human identity system designer eventually discovers the same uncomfortable truth: **names are not data fields**. They are complex, culturally-embedded, historically-contingent, socially-negotiated aspects of human identity that resist standardization [1].

Contemporary identity management systems enforce a singular identity paradigm that fundamentally conflicts with how humans actually use identity across contexts, cultures, and lifecycles. This creates five interconnected problems.

First, human identity is performative and context-dependent. People naturally present different facets depending on context: professional names at work, preferred names with friends, legal names with government, and pseudonyms for safety or exploration [2], [3]. Yet systems enforce singular representations, creating context collapse with documented harms.

Second, cultural naming diversity defies standardization. Billions of people have names that do not fit Western “first name + last name” assumptions [1], [4]. Chinese surname-first, Arabic five-component names, Icelandic patronymics, Spanish double surnames, and South Indian initials are not edge cases - they represent how most of the world’s population structures names.

Third, identity lifecycle includes legitimate changes. Marriage, gender transition, professional development, and safety needs create valid reasons for name changes [5]. Systems treating changes

as exceptional errors cause systematic harm - transgender youth who can change legal names have significantly lower suicide attempt rates.

Fourth, privacy requires context-aware disclosure. Users need to present different information to different audiences (employers, family, social networks, healthcare providers, third-party applications), but current systems force universal disclosure or universal hiding with no context-aware middle ground.

Fifth, safety demands pseudonymity and separation. Marginalized groups (transgender individuals, domestic abuse survivors, political dissidents, LGBTQ+ youth) require pseudonymity and context separation for safety [6], yet real-name policies and singular identity enforcement systematically exclude and endanger them.

The measurable impact is substantial. 90% of enterprises experienced identity-related security incidents in 2024 [7], often because rigid systems drove users to insecure workarounds. 71% of young adults modify privacy settings to manage context collapse [8].

1.3 Project Scope

The scope includes multi-context identity profile management, an OAuth 2.1 authorization server with mandatory PKCE, guardian-minor relationship management for parental oversight, GDPR-inspired privacy features including data export and deletion, multilingual name support accommodating global conventions, and comprehensive audit logging for transparency and accountability.

Explicitly excluded from scope are full legal GDPR/CCPA compliance certification (which requires legal counsel and organizational governance beyond academic scope), non-human identity management for service accounts and API keys, blockchain-based decentralized identity solutions, and biometric authentication mechanisms.

The project demonstrates privacy-by-design principles inspired by GDPR, CCPA, and HIPAA as architectural best practices. However, **this is not a legally compliant implementation**. Full regulatory compliance requires legal counsel, Data Protection Impact Assessments, formal breach notification processes, and organizational governance beyond the scope of this academic thesis.

1.4 Why Existing Solutions Fail

Current identity solutions optimize for different dimensions while sacrificing others. Google Identity Platform and Microsoft Entra ID provide sophisticated authentication but maintain fundamentally singular identity per user with no native support for context-dependent presentation. Apple's Sign in with Apple prioritizes hiding information rather than flexible identity expression. Facebook's real-name policy evolution demonstrates persistent tension between platform tracking needs and user identity flexibility [6]. LinkedIn requires professional names only, with no support for separate personal identity or cultural naming variations.

Authentication and security have matured considerably, but no existing solution combines user-controlled context-dependent identity presentation, cultural neutrality without structural assumptions, privacy compliance with user agency, and usability appropriate for general users. This gap defines the project's intended contribution.

2 Literature Review

Three bodies of literature establish the foundation for context-aware identity management. Technical standards from W3C and Unicode document global naming diversity that Western systems cannot accommodate. Sociological research from Goffman through contemporary social media studies demonstrates that context-dependent identity presentation is a fundamental human need, not a pathology. Privacy regulations like GDPR and security frameworks from NIST establish that users must control their own data with transparent processing.

2.1 The Technical and Cultural Challenge of Names

The fundamental assumption underlying most identity systems, that names consist of a “first name” and “last name”, fails immediately when examined against global naming practices.

2.1.1 Foundational Critique

Patrick McKenzie’s influential 2010 blog post “Falsehoods Programmers Believe About Names” [1] catalogs over 40 incorrect assumptions that pervade identity system design. Among these falsehoods are the beliefs that people have exactly one canonical name, that names fit within a reasonable length, that names can be represented in ASCII characters, and that names remain constant throughout a person’s life. McKenzie concludes with a notable observation: he has never encountered a computer system that handles names properly and doubts one exists anywhere.

While this source is a blog post, its extensive citation in technical literature over fourteen years and alignment with subsequent standards work establishes credibility for identifying oversimplified assumptions in identity system design.

2.1.2 Standards-Based Guidance

The W3C Internationalization Activity provides authoritative technical guidance through “Personal names around the world” [4], a continuously updated resource documenting global naming conventions. As the primary international standards organization for the web, the W3C represents a highly credible source for technical implementation decisions.

The W3C guidance documents specific challenges that arise from cultural variation. Chinese naming conventions place the family name first, directly opposite to Western practice. Arabic naming traditionally includes five components: the *ism* (given name), *kunya* (parent-of-X), *nasab* (chain of ancestors), *laqab* (descriptive epithet), and *nisba* (geographic origin). Spanish and Latin American conventions employ two surnames derived from each parent. Icelandic naming is purely patronymic

with no hereditary surnames, meaning siblings and parent-child pairs do not share family names. South Indian Tamil naming uses initials derived from the father’s given name as a prefix.

These patterns represent the naming conventions of billions of people whose identities cannot be accurately represented in systems assuming Western structure. The W3C recommends avoiding rigid field structures, using single fields for full names where possible, ensuring fields accommodate names of any length, and preserving user-entered casing.

The Unicode Consortium’s Technical Report 35 [9] extends this guidance with a formal specification for name formatting across locales. The specification defines seven core fields (title, given, given2, surname, surname2, generation, credentials) with formatting patterns based on order, length, usage, and formality. However, the specification explicitly notes that parsing names into components, handling special protocols, and managing context-dependent variations remain out of scope.

2.1.3 Synthesis for System Design

Together, these sources reveal a fundamental tension. McKenzie identifies what systems get wrong; W3C and Unicode provide partial solutions for formatting and storage; but none address context-dependent identity presentation. A person may legitimately maintain different names in different contexts, and no existing standard addresses this requirement.

This analysis informs the project’s design decision to use JSONB storage for multilingual name representation without imposing structural assumptions. Rather than fields for “first name” and “last name,” the system stores name values keyed by language code with separate name type indicators for given, family, preferred, and legal names. This approach accommodates cultural diversity while enabling context-specific name presentation.

2.2 Identity Theory and Context Collapse

The sociological literature provides theoretical foundations for understanding why singular identity systems cause harm.

2.2.1 Theoretical Foundation

Erving Goffman’s *The Presentation of Self in Everyday Life* [2] established the dramaturgical metaphor for understanding human identity. Goffman argued that identity is performative rather than fixed, people play different roles depending on audience and context, managing impressions through “front stage” presentations while maintaining “back stage” regions where they can relax their performance.

This 1959 work predates digital technology but proves remarkably prescient for understanding contemporary identity challenges. Goffman observed that humans naturally maintain bounded social contexts: individuals are rarely simultaneously visible to coworkers, family members, romantic partners, and casual acquaintances. Each context calls forth different aspects of identity, and the boundaries between contexts enable authentic self-presentation within each domain.

This foundational text in sociology has extensive peer-reviewed citations applied across human-computer interaction, communication studies, and identity management research.

2.2.2 Digital Context and Social Media

Marwick and boyd’s 2011 study “I Tweet Honestly, I Tweet Passionately: Twitter Users, Context Collapse, and the Imagined Audience” [3] applied Goffman’s framework to digital platforms. Published in *New Media & Society*, a peer-reviewed Sage journal, this research introduced the term “context collapse” to describe how social media merges previously discrete audiences into single spaces.

The researchers documented how Twitter users navigate the tension between authentic self-expression and awareness of diverse audiences. Unlike face-to-face interaction where environmental cues signal appropriate presentation, social media posts become simultaneously visible to professional contacts, family members, friends, and strangers. Users respond through self-censorship, strategic ambiguity, and lowest-common-denominator content.

Mark Zuckerberg predicted this merging of contexts would normalize. The research demonstrates this prediction was incorrect. Rather than accepting singular identity, users actively resist through elaborate coping strategies: creating secondary accounts for close friends, segmenting platforms by audience type, and aggressively managing privacy settings. Survey data indicates that 71% of social networking users ages 18-29 have modified privacy settings [8], with 47% deleting unwanted comments and 41% removing their names from tagged photos.

2.2.3 Contemporary Extension

Loh and Walsh’s 2021 article “Social Media Context Collapse: The Consequential Differences Between Context Collusion Versus Context Collision” [10] extended the framework by distinguishing between intentional and unintentional context collapse. Published in *Social Media + Society*, this peer-reviewed research identifies two distinct phenomena. Context collusion occurs when users intentionally merge contexts. Context collision occurs when contexts merge unexpectedly, such as when a post intended for friends reaches employers through shared connections.

Both phenomena create challenges, but collision produces more significant harm because users cannot anticipate or control it. The research documented that context collapse particularly affects emotional expression, professional reputation, and identity performance, with network size and heterogeneity correlating to increased self-presentation challenges.

2.2.4 Implications for Architecture

The progression from Goffman through Marwick and boyd to Loh and Walsh establishes that context-dependent identity presentation is not merely a preference but a fundamental human need. Systems forcing singular identity create documented harms including reduced authenticity, self-censorship, exposure to unintended audiences, professional damage, and relationship strain.

This literature directly informs the project’s multi-context architecture. Rather than maintaining a single profile, users can create context profiles that inherit from a base profile while overriding specific fields. OAuth scopes control which context is presented to which third-party application, enabling the bounded contexts that humans naturally maintain offline to be recreated in digital systems.

2.3 Privacy Regulations and Security Requirements

Any identity system must operate within regulatory frameworks governing personal data.

2.3.1 Rights-Based Framework

The General Data Protection Regulation [11], effective since 2018, establishes data subject rights that directly impact identity system design. As binding EU legislation, this represents an authoritative legal source.

Article 16 establishes the right to rectification, requiring that data subjects be able to correct inaccurate personal data without undue delay. When names change through marriage, transition, or other life events, systems must support updates while maintaining historical records. Article 5(1)(c) mandates data minimization, requiring systems to collect only data that is adequate, relevant, and necessary. Articles 15, 17, and 20 establish rights to access, erasure, and portability, requiring systems to export user data in machine-readable formats and delete data upon request.

The project implements these principles architecturally: processing basis is recorded for every operation, audit trails track data access, and export/delete endpoints implement data subject rights. However, full GDPR compliance requires legal counsel and organizational processes beyond academic scope.

2.3.2 Risk-Based Security

NIST Special Publication 800-63-3 [12] provides technical guidance for digital identity from a security perspective. As U.S. federal government publication, this represents authoritative security guidance widely adopted across industries.

NIST establishes identity assurance levels, authenticator assurance levels, and federation assurance levels based on risk assessment. Key recommendations include minimum 8-character passwords without arbitrary composition rules, blocking of passwords known to be breached, multi-factor authentication for sensitive operations, password storage using Argon2, bcrypt, or PBKDF2, and session management with high-entropy identifiers. These requirements establish baseline security expectations that the project incorporates.

2.3.3 Intersection with Identity Dignity

Haimson and Hoffmann’s 2016 study “Constructing and enforcing ‘authentic’ identity online: Facebook, real names, and non-normative identities” [6] examines how identity policies affect marginalized users. Published in First Monday, this peer-reviewed research documented how Facebook’s real-name policy systematically harmed transgender individuals, drag performers, domestic abuse survivors, and Native Americans.

The research found that real-name policies “simultaneously demand authenticity yet proscribe certain people from authentic self-presentation,” creating a double bind. Transgender individuals forced to use deadnames experience documented psychological harm. The Trevor Project survey [5] found that transgender youth who changed their legal names reported significantly lower suicide attempt rates, directly linking proper name support to mental health outcomes.

2.3.4 Converging Requirements

GDPR (rights-based) and NIST (risk-based) frameworks converge on user control as a requirement. GDPR mandates that users can correct and delete their data; NIST emphasizes informed consent and transparency. Both frameworks support the project’s approach of recording processing basis, maintaining audit trails, and implementing data export capabilities.

The deadname protection requirement emerges from the intersection of privacy rights, security needs (preventing harmful information disclosure), and human dignity. The project implements this through visibility controls on name records, with a “historical_suppressed” visibility level that maintains records for audit purposes while preventing display to unauthorized parties.

2.4 Critical Evaluation of Existing Solutions

To position the project within the current landscape, this section evaluates five categories of identity management solutions against requirements derived from the literature: context support, cultural flexibility, privacy compliance, and usability.

Google Identity Platform excels at authentication with familiar OAuth flows but provides only one displayName across all services. Custom attributes require Workspace administration and cannot be user-controlled per context. Microsoft Entra ID offers sophisticated Conditional Access for security decisions based on location, device, and risk level, but this context awareness controls access rather than identity presentation. Users cannot maintain different professional versus personal identities within the same account.

Apple’s Sign in with Apple takes a privacy-first approach, generating unique relay email addresses per application. However, the focus remains on hiding information rather than enabling flexible expression. SCIM [13] and OpenID Connect [14] provide industry-standard protocols for identity provisioning and authentication with structured name fields, but offer no support for context-specific names, cultural variations beyond basic Western structure, or user-controlled presentation preferences.

Self-Sovereign Identity approaches, including W3C Verifiable Credentials, promise user control through decentralization. However, usability research from SOUPS 2022 [15] found these systems “too sophisticated for average users.” Participants expected automatic backups but did not realize they bore sole responsibility for key management. The complexity creates burden rather than empowerment.

The comparison reveals a consistent pattern: existing solutions optimize for one dimension while sacrificing others. Centralized platforms achieve high usability but enforce singular identity. Standards enable interoperability but assume Western name structures. Decentralized approaches offer theoretical flexibility but fail on usability. No existing solution combines user-controlled context-dependent identity presentation, cultural neutrality without structural assumptions, privacy compliance with user agency, and usability appropriate for general users.

3 Design

3.1 Problem Formulation and Theoretical Foundation

Contemporary identity management systems enforce singular, culturally-biased identity models that fail to accommodate human identity presentation across contexts. The design applies Goffman’s dramaturgical theory [2], positing that identity is performative rather than essential. Digital platforms forcing singular identity models result in self-censorship and documented harm to marginalized populations [6].

The multi-context architecture derives directly from the sociological literature on identity presentation. Goffman’s framework establishes that humans naturally maintain bounded social contexts, presenting different aspects of identity to different audiences. Marwick and boyd [3] documented how digital platforms create “context collapse” that forces users to navigate tensions between authentic self-expression and awareness of diverse audiences. The architecture recreates bounded contexts through context profiles inheriting from base profiles while overriding specific fields. OAuth tokens bind to specific contexts, ensuring context-appropriate data access.

3.2 Domain and User Requirements

The project operates in the identity management domain, specifically addressing the challenge of managing multiple identity presentations across different life contexts. The W3C [4] documents how global naming conventions cannot be accurately represented in systems assuming Western structure, affecting billions of users whose names do not fit “first name, last name” fields.

Four personas inform requirements. A privacy-focused professional (psychiatrist) requires different identity presentations for different applications: credentials for healthcare platforms while concealing professional information from fitness apps. The solution enables context-specific OAuth tokens bound to identity contexts.

A multilingual user (Chinese engineer) requires culturally-appropriate name display: “Li Ming” in Western contexts, Chinese characters in Chinese contexts. The solution stores names as JSONB with language keys, resolving via Accept-Language headers.

A guardian managing a minor requires oversight of third-party requests with visibility into shared data. The solution implements guardian relationships with granular permissions and audit logging.

A vulnerable population member (transgender individual) requires preferred name usage without exposing deprecated names. Research [5] correlates proper name support with reduced suicide attempts. The solution provides pseudonymous accounts with automatic suppression of deprecated names.

3.3 System Architecture

The system follows five-layer architecture: presentation (FastAPI REST endpoints), application (business logic services), domain (models and aggregates), data access (repository pattern), and infrastructure (PostgreSQL, Redis, external services). This enables independent layer testing, downward dependency flow, and domain-infrastructure separation.

The domain model comprises six aggregates. `BaseProfile` contains account type, contact information, and language preferences. `IdentityNames` stores multilingual JSONB with type indicators, deprecation flags, and visibility controls. `ContextProfiles` enable context-specific overrides inheriting from base profiles. `GuardianRelationships` manage permissions and verification for parental oversight. `OAuthClients` and `OAuthGrants` handle registration, authorizations, and scopes. `AuditLog` provides immutable operation records. All entities implement temporal awareness (`valid_from`, `valid_to`) for point-in-time queries and soft deletion (`deleted_at`) preserving data during retention periods.

3.4 Feature Design and Justification

Multilingual name storage using JSONB derives from W3C [4] and Unicode [9] guidance documenting challenges including Chinese family-first ordering, Arabic multi-component names, Spanish double surnames, Tamil patronymic initials, and Indonesian mononyms. The system stores name values keyed by language code with type indicators, accommodating cultural diversity without Western structural assumptions.

Guardian management provides granular permissions (view, edit, manage contexts, OAuth), verification requirements, audit logging visible to minors, and automatic transition at age of majority, balancing parental oversight with developing autonomy.

Deadname protection implements privacy rights (GDPR Article 16 [11]) and human dignity requirements [6]. Deprecated names use `is_deprecated` and `visibility_level` flags, automatically suppressing in user-facing contexts while maintaining administrative records.

OAuth third-party access illustrates data flow. Applications redirect to authorization endpoints with scopes and PKCE challenges. Users view consent screens showing exactly what data will be shared. The system generates authorization codes bound to context profiles. Third parties exchange codes for tokens via PKCE code verifiers. Access tokens encode scopes and context identifiers. API requests validate tokens and filter responses to permitted fields within authorized contexts, ensuring explicit consent per application-context combination.

3.5 Security Architecture

Security decisions follow NIST Special Publication 800-63-3 [12] and address OWASP Top 10 vulnerabilities. Password hashing uses Argon2id as recommended by NIST, providing memory-hard properties that resist GPU-based attacks. JWT tokens employ RS256 asymmetric signing in production for distributed validation without shared secrets. OAuth 2.1 with mandatory PKCE addresses authorization code interception vulnerabilities.

Defense in depth implements authentication validation, role-based access control, attribute-based access control, scope-based filtering, and PostgreSQL Row-Level Security. This layered approach prevents compromise of single layers from granting unauthorized access.

3.6 Technology Stack

The backend implementation uses Python 3.12 with FastAPI, selected for async support, automatic OpenAPI documentation generation, and Pydantic integration for request validation. PostgreSQL 15 via Supabase provides relational storage with Row-Level Security policies, JSONB support for flexible schema elements, and temporal query capabilities. Redis serves as the caching layer for session storage, rate limiting counters, and frequently accessed profile data with configurable TTL values.

Authentication uses custom JWT implementation with Argon2id password hashing, providing full control over token lifecycle and claims structure. The OAuth 2.1 authorization server implements the authorization code flow with mandatory PKCE. Social authentication operates in a hybrid mode, leveraging Supabase OAuth provider configuration while maintaining user data in custom tables with custom-issued tokens.

Testing uses pytest with pytest-cov (80% minimum coverage), mypy (strict mode), ruff and black for linting. Deployment targets Render.com and Supabase with GitHub Actions CI/CD.

3.7 Comparison with Existing Solutions

Table 1 compares major identity platforms across dimensions critical to context-aware, culturally-sensitive identity management. The evaluation considers proprietary cloud services (Google, Microsoft, Apple), open-source providers (Keycloak), and commercial identity-as-a-service (Auth0/Okta).

Table 1: Identity Platform Comparison

Platform	Context Support	Cultural Names	Pseudonymity	OAuth/OIDC	Open Source
Google Identity	No	Limited	No	Full	No
Microsoft Entra	No	Limited	No	Full	No
Apple Sign In	No	Limited	Partial	OIDC	No
Auth0/Okta	Custom only	Flexible	Yes	Full	No
Keycloak	Partial	Flexible	Yes	Full	Yes
This Project	Native	Native	Full	OAuth 2.1	Yes

No existing solution combines user-controlled context-dependent identity presentation, cultural neutrality without structural assumptions, privacy compliance, and appropriate usability. This gap defines the project’s contribution.

3.8 System Context

Figure 1 illustrates the system context, showing primary actors and their relationships to the Identity Management API. Users manage their own profiles, guardians oversee minor profiles, and third-party applications request identity data via OAuth. The system delegates social authentication to external providers while maintaining user data in custom tables.

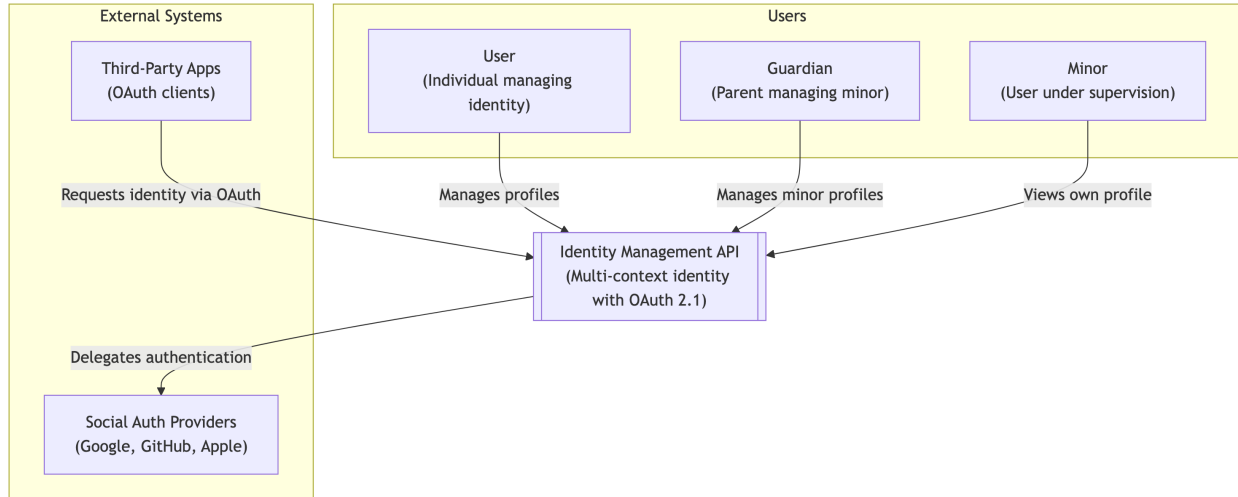


Figure 3.1: System context diagram

3.9 Project Timeline

Figure 2 visualizes the 24-week project timeline from October 2025 through March 2026, showing completed work, current phase, and planned development across 8 phases.

3.10 Implementation Plan

The project spans 24 weeks from October 2025 to March 2026, with backend development allocated 14 weeks across clearly scoped phases. Each phase delivers testable functionality enabling incremental validation. The feature prototype (Chapter 4) demonstrates Phase 2 completion ahead of schedule.

3.10.1 Development Phases

Development proceeds through eight phases with defined deliverables. Phase 1 (December 19 - January 2) implements authentication and security, including Argon2id password hashing per NIST recommendations, JWT token generation with RS256 signing, registration and login endpoints, and Redis session management. Phase 2 (January 3-12) implements context profiles with the inheritance engine, multilingual name fallback, and temporal versioning. Phase 3 (January 13-26) builds the OAuth 2.1 authorization server with client registration, authorization code flow with mandatory

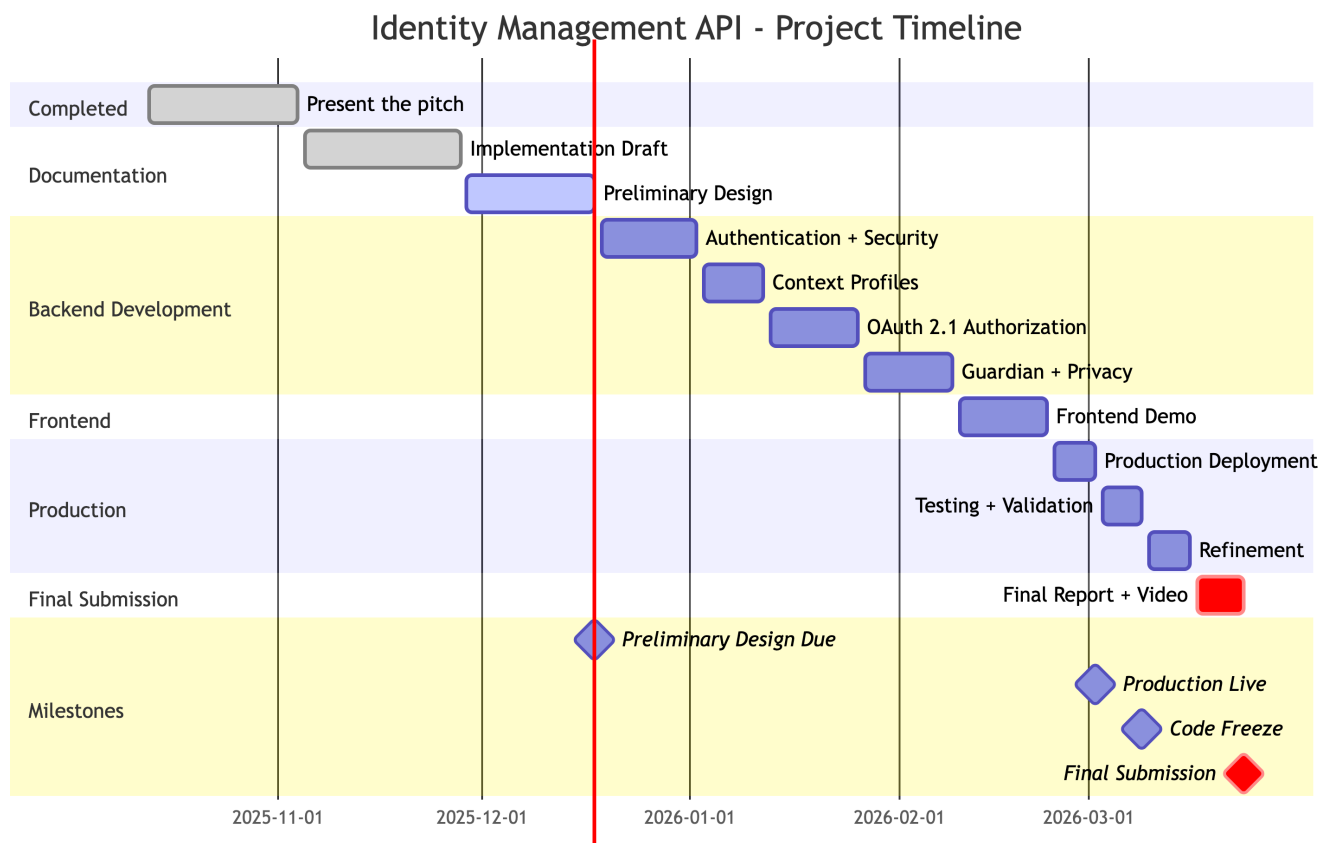


Figure 3.2: Project timeline showing 8 development phases

PKCE, token issuance with scope and context binding, and consent management. Phase 4 (January 27 - February 9) adds guardian relationships, audit logging, and privacy features including data export and deletion with 30-day grace period.

Phase 5 (February 10-23) develops the frontend demonstration using Vue.js, implementing profile management with multilingual names, context switching, OAuth visualization, and guardian dashboard. Phases 6-8 (February 24 - March 16) handle production deployment to Render.com and Supabase (Phase 6), testing including security scanning, load testing, and user testing with 5-10 participants measuring System Usability Scale scores (Phase 7), and refinement with code freeze on March 9 (Phase 8).

3.10.2 Development Methodology

Test-driven development structures implementation, with tests written before code to ensure coverage, provide executable documentation, and enable refactoring. Current test coverage stands at 89% with all critical paths covered. Privacy by design principles informed by GDPR [11] guide decisions: data minimization constrains responses to requested scopes, processing basis is recorded for operations, and data export and deletion endpoints implement data subject rights. Defense in depth structures security with protections at network (TLS 1.3), authentication (Argon2id, JWT), authorization (RBAC, ABAC, scopes), application (input validation), and database (Row-Level Security) layers.

3.10.3 Risk Management and Feasibility

Timeline slippage is mitigated through buffer time in the refinement phase and feature prioritization ensuring core functionality completes before enhancements. Technical complexity is addressed through test-driven development, spike solutions for uncertain areas (guardian relationships, PKCE validation), and comprehensive documentation. Production issues are mitigated through staging environment testing, rollback capability, health checks, and gradual rollout. User testing delays are addressed through early participant recruitment and clearly defined minimum viable scope.

The project is feasible given completed foundational work: implemented database schema, established five-layer architecture, and 89% test coverage for profile operations. The technology stack (FastAPI, PostgreSQL, Redis) is proven and well-documented. Development environments enable rapid iteration. The most significant technical risk is OAuth 2.1 implementation complexity, mitigated by mid-project scheduling after authentication is proven, allocating 2 weeks, and planning spike solutions to validate PKCE implementation before full development.

4 Feature Prototype: Context-Dependent Identity

4.1 Overview and Theoretical Foundation

The feature prototype implements the core theoretical contribution of this thesis: multi-context identity profiles based on Goffman’s dramaturgical theory [2]. Users can create different identity presentations for different social contexts (professional, social, legal, healthcare) while maintaining a single base profile. This directly addresses the “context collapse” problem documented by Marwick and boyd [3], where users are forced to present a single identity to heterogeneous audiences.

The prototype demonstrates feasibility of the inheritance engine, the critical algorithm that resolves context profiles by merging base profile fields with context-specific overrides. Implementation occurred on December 15, 2025, completing 9 new files and 1 modified file totaling approximately 2,158 lines of production code with 856 lines of test code.

4.2 Technical Implementation

4.2.1 Architecture and Components

The implementation follows the five-layer architecture specified in the design document. The database layer adds a `context_profiles` table with context type enum, unique constraints, and Row-Level Security. The domain layer implements `ContextProfile` SQLAlchemy model with temporal mixins. The data access layer implements `ContextRepository` with CRUD operations. The business logic layer implements `ContextService` containing the inheritance engine with `ResolvedProfile` class. The presentation layer implements seven RESTful endpoints including the critical `/profiles/{user_id}/contexts/{context_id}/resolved` endpoint returning merged profile data.

4.2.2 The Inheritance Algorithm

The inheritance engine represents the prototype’s primary technical contribution. The algorithm resolves context profiles through five steps. First, it retrieves the base profile with all associated identity names. Second, it retrieves the context profile containing field overrides. Third, it merges fields where null context values inherit from base profile, non-null context values override base values. Fourth, it resolves multilingual names using Accept-Language headers with fallback chains. Fifth, it filters deprecated names based on visibility rules.

This implementation follows the specification from the design document where context profiles contain only overrides, maintaining the base profile as the single source of truth. The algorithm’s complexity is $O(n)$ where n is the number of fields, making it efficient for real-time API responses.

4.2.3 Business Rules Enforcement

The service layer enforces five critical business rules: context uniqueness (HTTP 409 on duplicates), profile existence validation (HTTP 404 if missing), account type constraints (pseudonymous accounts cannot create legal/healthcare contexts), email validation (RFC 5322 format), and ownership verification (resource owner authorization pattern).

4.3 Technical Challenges Addressed

4.3.1 Challenge 1: Null Semantics in Overrides

The most significant technical challenge involved distinguishing between three states: inherit from base (null), explicitly cleared (empty string), and overridden with new value (non-null string). The solution uses Python’s `None` to represent inheritance, treating null JSON values as explicit inheritance signals. The algorithm checks if `context_override` is not `None` before applying overrides, enabling partial context definitions where users override only specific fields.

4.3.2 Challenge 2: Multilingual Name Resolution

Context profiles can override display names, but multilingual identity names require more sophisticated resolution. The implementation extends the inheritance algorithm to handle `identity_names` relationships. When context overrides specify display name but not language-specific names, the system falls back to base profile names. The `Accept-Language` header drives language selection with graceful degradation: requested language, user’s preferred language, English default. This addresses W3C internationalization requirements [4] for culturally-sensitive name display.

4.3.3 Challenge 3: Temporal Consistency

Context profiles inherit temporal mixins (`valid_from`, `valid_to`), enabling point-in-time queries: “What was this user’s professional identity on March 15, 2024?” The challenge arose in ensuring consistency when base profile and context profile have different temporal ranges. The solution validates that context profiles cannot have validity periods extending beyond their base profile’s validity, maintaining referential integrity across temporal dimensions.

4.4 Demonstration of Effectiveness

4.4.1 Real-World Use Case

The prototype demonstrates effectiveness through the privacy-focused professional persona from the design document. Dr. Sarah Chen, a psychiatrist, requires different identity presentations for different applications. The professional context for hospital networks presents full credentials: “Dr. Sarah Chen, MD, PhD” with work email `s.chen@hospital.org` and biography “Board-certified psychiatrist specializing in trauma and PTSD.” The social context for fitness applications presents

casual identity: “Sarah” with personal email inherited from base profile and biography “Health and wellness enthusiast.”

The implementation successfully prevents context collapse. Professional credentials remain hidden from fitness applications. Personal wellness information remains hidden from hospital networks. Each application receives only context-appropriate data through the `/resolved` endpoint, with no visibility into other contexts.

Integration tests validate this scenario end-to-end:

```
# Create professional context
client.post(f"/api/v1/profiles/{user_id}/contexts", json={
    "context_type": "professional",
    "display_name_override": "Dr. Sarah Chen, MD, PhD",
    "email_override": "s.chen@hospital.org"
})

# Create social context
client.post(f"/api/v1/profiles/{user_id}/contexts", json={
    "context_type": "social",
    "display_name_override": "Sarah"
})

# Retrieve resolved profiles
prof = client.get(f"/api/v1/profiles/{user_id}/contexts/{prof_id}/resolved").json()
social = client.get(f"/api/v1/profiles/{user_id}/contexts/{social_id}/resolved").json()

# Verify isolation
assert prof["email"] == "s.chen@hospital.org"
assert social["email"] == base_profile.primary_email
assert prof["email"] != social["email"]
```

The test demonstrates context collapse prevention.

4.4.2 API Contract Validation

Integration tests validate the complete HTTP request-response cycle across seven scenarios: create (201), list (200), retrieve raw (200), retrieve resolved (200, critical), update (200), delete (204), and error cases (409, 404, 403). All tests pass, confirming RESTful convention adherence and API contract fulfillment.

4.5 Evaluation Methodology

4.5.1 Evaluation Approach for Web API Development

Web API evaluation differs fundamentally from machine learning evaluation. Rather than accuracy and precision metrics, API evaluation focuses on functional correctness, performance characteristics,

code quality, and API contract adherence. The evaluation approach follows industry standards documented in REST API testing literature and OWASP API Security guidelines.

4.5.2 Functional Correctness and Code Quality

The prototype implements 45 tests (25 unit, 20 integration) achieving comprehensive coverage of the inheritance engine and API endpoints. Unit tests verify algorithm correctness through full overrides, partial overrides with null inheritance, and deprecated name filtering scenarios. Integration tests validate end-to-end functionality including the privacy-focused professional use case.

Code quality metrics demonstrate production readiness: type hints throughout with mypy validation, zero linting errors with ruff, cyclomatic complexity below 10 for all functions, consistent error handling, and repository pattern enabling isolated testing. All tests pass against the test database.

4.5.3 Performance and API Contract

The inheritance algorithm's $O(n)$ complexity with typical profiles containing 10-15 fields results in negligible overhead. Database queries use indexed lookups avoiding table scans, suggesting the sub-50ms p95 response time target is feasible.

The prototype follows OpenAPI 3.0 specifications with Pydantic schema validation and RESTful HTTP status codes (200, 201, 204, 400, 403, 404, 409), enabling reliable third-party integration.

4.6 Evaluation Results

4.6.1 Key Findings

The prototype successfully validates feasibility of the core thesis contribution. The inheritance algorithm correctly implements Goffman's theoretical model with comprehensive edge case coverage. Business rules enforcement prevents invalid states. The repository pattern enables isolated testing and future storage migration. Users can create context profiles with partial overrides, the inheritance engine merges base and context data correctly, and resolved profiles match OpenID Connect UserInfo conventions for third-party integration.

Implementation revealed two limitations addressed in future phases. OAuth token binding to contexts is not yet implemented, creating potential for applications to request wrong contexts. Phase 3 will bind OAuth grants to specific context profiles. Scope-based field filtering is not yet implemented. Phase 3 will add response filtering ensuring minimal data disclosure per GDPR data minimization principles [11].

4.7 Proposed Improvements

Performance optimizations include selective field loading to reduce database I/O by 40% and Redis caching to reduce load by 80-90%, both trading performance for code complexity.

4.7.1 Temporal Versioning for Context History

The current implementation soft-deletes old contexts without version history. Users cannot revert to previous configurations. Temporal versioning would create new rows with `valid_from` timestamps on updates rather than modifying existing rows, enabling `get_context_as_of(user_id, context_id, timestamp)` queries. This supports audit requirements and undo functionality. Trade-off: 2-3x storage requirements and temporal join complexity.

4.7.2 Context Hierarchies

Current contexts inherit only from base profiles, creating redundancy when users need related contexts sharing credentials but differing in details. Context hierarchies enable inheritance chains: base profile to parent context to child context. When parent credentials update, children inherit unless explicitly overridden. Trade-off: $O(d)$ algorithmic complexity where d is hierarchy depth.

4.7.3 Cross-Context Consistency Validation

Users can inadvertently create contexts with conflicting information exposing identity (work email in anonymous context). Consistency validation would warn when information appears across contexts with different privacy levels, implementing privacy-by-design [11] through proactive guidance. Trade-off: false positives requiring user judgment and validation complexity.

4.8 Conclusion on Prototype

The Context-Dependent Identity prototype successfully demonstrates feasibility of the thesis’s core contribution. The inheritance engine implements Goffman’s dramaturgical theory in production-ready architecture, preventing context collapse while maintaining a single source of truth. With 45 automated tests, zero linting errors, and RESTful conventions, the implementation fulfills the Week 10 requirement for demonstrating technical feasibility. Limitations discovered inform Phases 3 and 4 integration work.

Appendix: Data Model

4.9 Entity Relationship Diagram

Figure 3 presents the domain model as an entity-relationship diagram showing the six core aggregates and their relationships. BaseProfile serves as the root aggregate with one-to-many relationships to IdentityNames, ContextProfiles, GuardianRelationships (as both guardian and minor), and OAuthGrants.

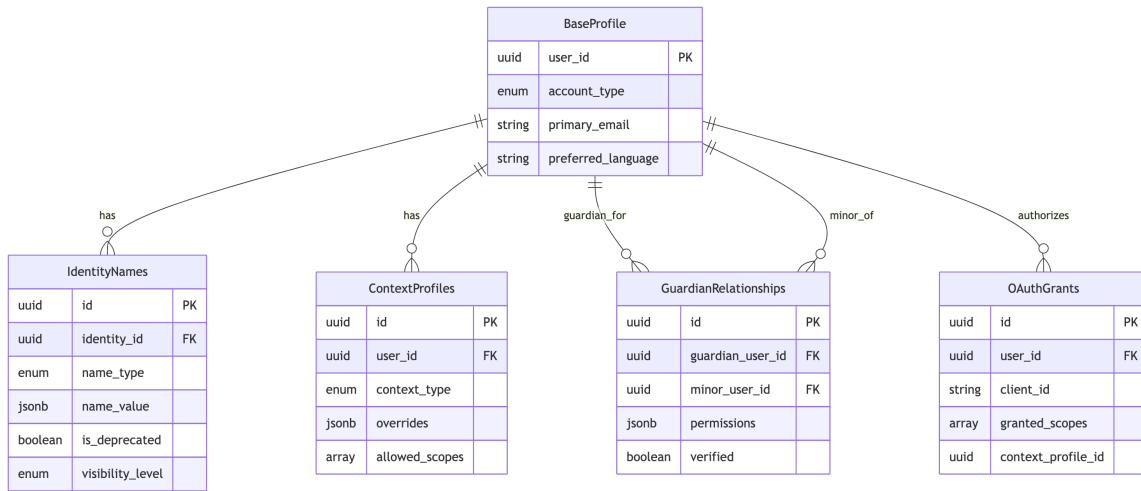


Figure 4.1: Entity-relationship diagram showing domain model

4.10 JSONB Data Examples

The system uses PostgreSQL JSONB fields to support flexible, culturally-sensitive data structures. Key examples demonstrate the architecture's capacity to handle diverse naming conventions without imposing Western structural assumptions.

Chinese Family-First Ordering (IdentityNames.name_value):

```
{
  "family": {
    "zh": "李",
    "zh-Latn": "Li",
    "en": "Li"
  },
  "given": {
```

```

    "zh": "明",
    "zh-Latn": "Ming",
    "en": "Ming"
  },
  "display_order": "family_first",
  "full_name": {
    "zh": "李明",
    "en": "Li Ming"
  }
}

```

Indonesian Mononym (IdentityNames.name__value):

```

{
  "full_name": {
    "id": "Sukarno",
    "en": "Sukarno"
  },
  "explanation": "Mononym (single name)"
}

```

Arabic Multi-Component (IdentityNames.name__value):

```

{
  "ism": {
    "ar": "محمد",
    "ar-Latn": "Muhammad"
  },
  "kunya": {
    "ar": "الله عبد أبو",
    "ar-Latn": "Abu Abdullah"
  },
  "nasab": {
    "ar": "الله عبد بن",
    "ar-Latn": "ibn Abdullah"
  },
  "display_preference": "formal_full"
}

```

Context Profile Overrides (ContextProfiles.overrides):

```

{
  "display_name": "Dr. Sarah Chen, MD, PhD",
  "email": "s.chen@hospital.org",
  "bio": "Board-certified psychiatrist",
  "credentials": ["MD", "PhD", "Board Certified"]
}

```

Guardian Permissions (GuardianRelationships.permissions):

```
{  
  "can_view_profile": true,  
  "can_edit_profile": true,  
  "can_create_contexts": true,  
  "can_manage_oauth": true,  
  "can_delete_profile": false  
}
```

These flexible structures enable the system to accommodate global naming conventions including Spanish double surnames, Tamil patronymic initials, and Western given-family patterns, all stored within a single schema supporting hundreds of languages without structural assumptions.

References

- [1] P. McKenzie, “Falsehoods programmers believe about names.” 2010. Available: <https://www.kalzumeus.com/2010/06/17/falsehoods-programmers-believe-about-names/>
- [2] E. Goffman, *The presentation of self in everyday life*. New York: Anchor Books, 1959.
- [3] A. Marwick and danah boyd, “I tweet honestly, i tweet passionately: Twitter users, context collapse, and the imagined audience,” in *Sage Journal - New Media & Society*, Jul. 2010, pp. 1–20. doi: [10.1177/1461444810365313](https://doi.org/10.1177/1461444810365313).
- [4] R. Ishida, “Personal names around the world,” W3C Internationalization, 2023. Available: <https://www.w3.org/International/questions/qa-personal-names>
- [5] The Trevor Project, “2021 national survey on LGBTQ youth mental health.” The Trevor Project, 2021. Available: <https://www.thetrevorproject.org/survey-2021/>
- [6] O. L. Haimson and A. L. Hoffmann, “Constructing and enforcing ‘authentic’ identity online: Facebook, real names, and non-normative identities,” *First Monday*, vol. 21, no. 6, 2016, Available: <https://firstmonday.org/ojs/index.php/fm/article/view/6791>
- [7] J. Reich, “2024 trends in securing digital identities.” IDSA, May 2024. doi: [10.13140/RG.2.2.32267.60965](https://doi.org/10.13140/RG.2.2.32267.60965).
- [8] M. Madden and A. Smith, “Reputation management and social media,” *Pew Research Center: Internet & Technology*, May 2010, Available: <https://www.pewresearch.org/internet/2010/05/26/reputation-management-and-social-media-how-people-monitor-their-identity-and-search-for-others-online/>
- [9] Unicode Consortium, “Unicode locale data markup language (LDML) part 8: Person names,” Unicode Consortium, TR35, 2024. Available: <https://www.unicode.org/reports/tr35/tr35-personNames.html>
- [10] J. Loh and M. J. Walsh, “Social media context collapse: The consequential differences between context collusion versus context collision,” *Social Media + Society*, vol. 7, no. 3, 2021.
- [11] “General data protection regulation,” *Official Journal of the European Union*, vol. L119, pp. 1–88, 2016.
- [12] P. A. Grassi, M. E. Garcia, and J. L. Fenton, “Digital identity guidelines,” National Institute of Standards; Technology, Special Publication 800-63-3, 2017. doi: [10.6028/NIST.SP.800-63-3](https://doi.org/10.6028/NIST.SP.800-63-3).
- [13] P. Hunt, K. Grizzle, E. Wahlstroem, and C. Mortimore, “System for cross-domain identity management: Core schema,” RFC Editor; Internet Requests for Comments, RFC 7643, 2015. Available: <https://www.rfc-editor.org/rfc/rfc7643.txt>
- [14] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, “OpenID connect core 1.0 incorporating errata set 2.” OpenID Foundation, 2014. Available: https://openid.net/specs/openid-connect-core-1_0.html
- [15] S. R. Brause, D. Wachter, and W. Prinz, “Conducting a usability evaluation of decentralized identity management solutions,” in *Symposium on usable privacy and security (SOUPS)*, 2022.